

Neural Network - A Simple Perceptron Assignment

Question 1 What is Deep Learning? Briefly describe how it evolved and how it differs from traditional machine learning.

Answer:

Deep Learning is a subset of machine learning that uses artificial neural networks with multiple layers to learn patterns from data automatically.

Evolution:

- Began with simple neural networks (Perceptron in 1950s)
- Progress slowed due to limited computation
- Revived with big data, GPUs, and better algorithms
- Now widely used in AI fields like image recognition and NLP

Difference from Traditional ML:

Traditional ML	Deep Learning
Manual feature extraction	Automatic feature learning
Works on small datasets	Requires large datasets
Simpler models	Complex deep networks

Question 2 Explain the basic architecture and functioning of a Perceptron. What are its limitations?

Answer:

A Perceptron is a single-layer neural network used for binary classification.

Architecture:

- Inputs ($x_1, x_2, \dots$)
- Weights ($w_1, w_2, \dots$)
- Bias (b)
- Output:
 $y = \text{activation}(w \cdot x + b)$

Working:

1. Multiply inputs by weights
2. Add bias

3. Apply activation function

4. Generate output

Limitations:

- Cannot solve non-linear problems (e.g., XOR)
- Only binary classification
- Limited learning capability

Question 3 Describe the purpose of activation function in neural networks. Compare Sigmoid, ReLU, and Tanh functions.

Answer:

Activation functions introduce non-linearity in neural networks.

Comparison:

Function	Range	Advantages	Disadvantages
Sigmoid	(0,1)	Smooth output	Vanishing gradient
Tanh	(-1,1)	Zero-centered	Still vanishing gradient
ReLU	$[0,\infty)$	Fast, efficient	Dying ReLU problem

Question 4 What is the difference between Loss function and Cost function in neural networks? Provide examples.

Answer:

- **Loss Function:** Error for a single data point
- **Cost Function:** Average loss over dataset

Examples:

- Mean Squared Error (MSE)
- Binary Cross Entropy

Question 5 What is the role of optimizers in neural networks? Compare Gradient Descent, Adam, and RMSprop.

Answer:

Optimizers update weights to minimize loss.

Comparison:

Optimizer	Description
Gradient Descent	Basic but slow
RMSprop	Adaptive learning rate
Adam	Fast, combines momentum + RMSprop

Question 6 Write a Python program to implement a single-layer perceptron from scratch using NumPy to solve the logical AND gate.

Answer:

```
import numpy as np
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])
weights = np.zeros(2)
bias = 0
lr = 0.1
def step(x):
    return 1 if x >= 0 else 0
for epoch in range(10):
    for i in range(len(X)):
        linear = np.dot(X[i], weights) + bias
        y_pred = step(linear)
        error = y[i] - y_pred
        weights += lr * error * X[i]
        bias += lr * error
print("Weights:", weights)
print("Bias:", bias)
for i in X:
    print(i, "->", step(np.dot(i, weights) + bias))
```

```
Weights: [0.2 0.1]
Bias: -0.20000000000000004
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
```

Question 7 Implement and visualize Sigmoid, ReLU, and Tanh activation functions using Matplotlib.

Answer:

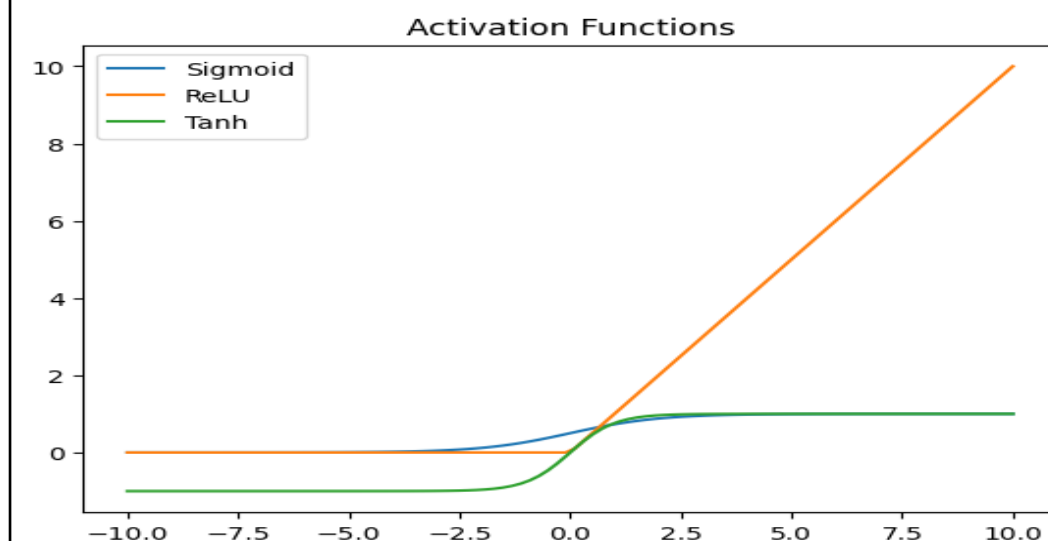
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)

sigmoid = 1 / (1 + np.exp(-x))
relu = np.maximum(0, x)
tanh = np.tanh(x)

plt.plot(x, sigmoid, label="Sigmoid")
plt.plot(x, relu, label="ReLU")
plt.plot(x, tanh, label="Tanh")
plt.legend()
plt.title("Activation Functions")
plt.show()
```

Output



Question 8 Use Keras to build and train a simple multilayer neural network on the MNIST digits dataset. Print the training accuracy.

Answer:

```
import tensorflow as tf

from tensorflow.keras import layers, models

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train, x_test = x_train/255.0, x_test/255.0

model = models.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=5)

print("Training Accuracy:", history.history['accuracy'][-1])
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 1s 0us/step
Epoch 1/5
1875/1875 ————— 8s 4ms/step - accuracy: 0.9248 - loss: 0.2597
Epoch 2/5
1875/1875 ————— 6s 3ms/step - accuracy: 0.9662 - loss: 0.1130
Epoch 3/5
1875/1875 ————— 8s 4ms/step - accuracy: 0.9770 - loss: 0.0779
Epoch 4/5
1875/1875 ————— 6s 3ms/step - accuracy: 0.9825 - loss: 0.0587
Epoch 5/5
1875/1875 ————— 7s 4ms/step - accuracy: 0.9863 - loss: 0.0457
Training Accuracy: 0.9862666726112366
```

Question 9 Visualize the loss and accuracy curves for a neural network model trained on the Fashion MNIST dataset. Interpret the training behavior.

Answer:

```
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train, x_test = x_train/255.0, x_test/255.0
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28,28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=5)
plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['accuracy'], label='Accuracy')
plt.legend()
plt.show()
```

Output

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 1s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 1s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
```

Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Epoch 1/5

1875/1875 ————— 8s 4ms/step - accuracy: 0.8241 - loss: 0.4982

Epoch 2/5

1875/1875 ————— 6s 3ms/step - accuracy: 0.8642 - loss: 0.3747

Epoch 3/5

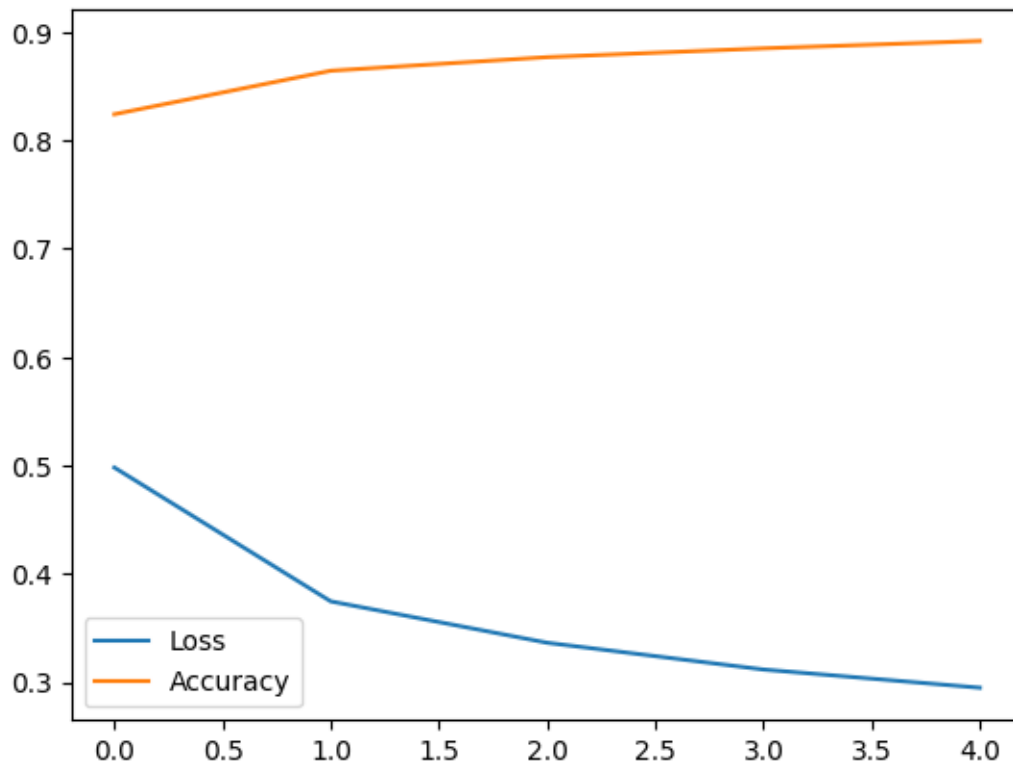
1875/1875 ————— 7s 4ms/step - accuracy: 0.8768 - loss: 0.3365

Epoch 4/5

1875/1875 ————— 6s 3ms/step - accuracy: 0.8849 - loss: 0.3118

Epoch 5/5

1875/1875 ————— 7s 4ms/step - accuracy: 0.8917 - loss: 0.2950



Question 10 You are working on a project for a bank that wants to automatically detect fraudulent transactions. The dataset is large, imbalanced, and contains structured features like transaction amount, merchant ID, and customer location. The goal is to classify each transaction as fraudulent or legitimate.

Explain your real-time data science workflow:

- How would you design a deep learning model (perceptron or multilayer NN)?
- Which activation function and loss function would you use, and why?
- How would you train and evaluate the model, considering class imbalance?
- Which optimizer would be suitable, and how would you prevent overfitting?

Answer:

Model Design:

- Use Multilayer Neural Network (Dense layers)
- Input → Hidden layers (ReLU) → Output (Sigmoid)

Activation Function:

- ReLU → hidden layers (fast)
- Sigmoid → output (binary classification)

Loss Function:

- Binary Cross-Entropy (best for classification)

Handling Imbalance:

- Use class weights
- Oversampling (SMOTE)
- Precision/Recall instead of accuracy

Optimizer:

- Adam (fast and adaptive)

Prevent Overfitting:

- Dropout layers
- Early stopping
- Regularization